

Nanobot — End-to-End Architecture

nanobot gateway · WebSocket ws://localhost:18790 · React 19 webapp (Vite :5174)

Browser · React 19 + TypeScript + Vite (port 5174)



```
WS Request: { type: "req", id: "r1", method: "chat.send", params: { sessionKey: "agent:main:webapp-<ts>", message: "..." } }
```

```
WS Response: { type: "event", event: "chat", payload: { state: "final", message: { role: "assistant", content: [...] } } }
```

```
{ type: "event", event: "agent", payload: { stream: "tool", data: { phase: "progress", message: "..." } } }
```

1 chat.send req

7 event frames

WebChannel · WebSocket Server ws://localhost:18790



Legend

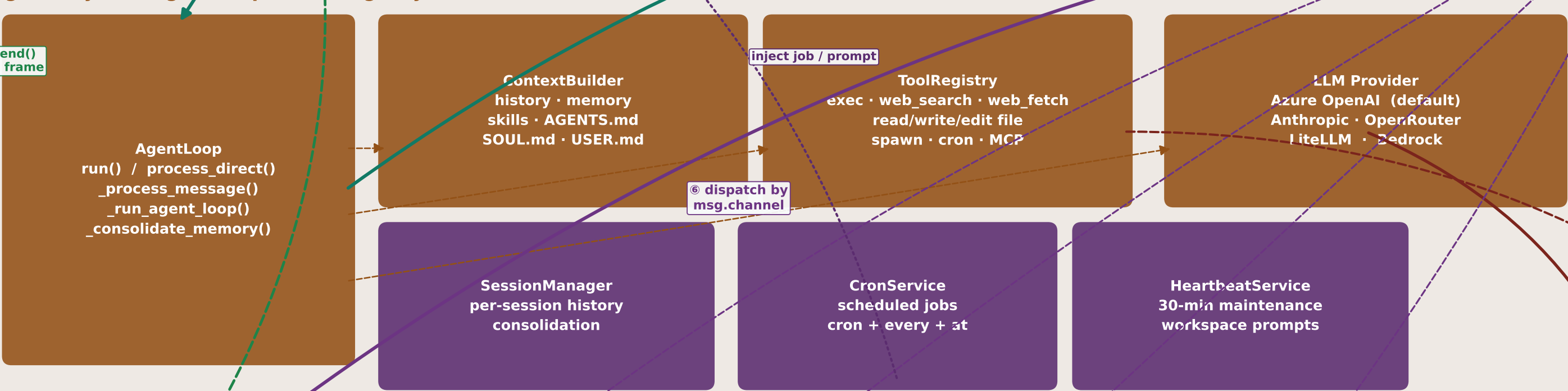
- WebSocket frame (browser ↔ gateway)
- Message Bus: InboundMessage / OutboundMessage
- Agent internal wiring
- ChannelManager dispatch
- External service call
- External platform API
- Cron / Heartbeat injection

Session Keys:
web → "agent:main:webapp-{ts}"
tg → "telegram:{chat_id}"
cli → "cli:direct"

Message Bus · async pub/sub (InboundMessage ↔ OutboundMessage)



Agent Layer · AgentLoop + ToolRegistry + LLM Provider + Sessions



ChannelManager · Outbound Router + External Services

